

Presence Detection with Ultra-Wide Band Radar

Matteo Figini¹, Tiya Jreige¹, and Valeria De Gennaro¹

¹Politecnico di Milano

August 28, 2025

1 Introduction

Ultra-Wide Band (UWB) radar excels in high-resolution sensing, material penetration, and multipath robustness, making it ideal for precise target detection in diverse environments. However, raw UWB data is often obscured by static clutter, necessitating advanced signal processing for accurate interpretation.

This project aims to establish an efficient pipeline for UWB radar target detection and localisation, specifically for real-time embedded system applications. By integrating sophisticated signal processing with modern machine learning, we seek to unlock UWB radar's full potential for autonomous perception.

The main goals of this project are:

- **Robust Signal Preprocessing and Spatial Enhancement:** To effectively suppress clutter and accurately determine target range and azimuth angle through de-cluttering and Digital Beamforming (DBF), generating interpretable Range-Azimuth images.
- **Automated Object Detection and Localization:** To automatically identify and localize targets using a meticulously labeled dataset of beamformed images. This is achieved via the lightweight FOMO (Fast Objects, Mobile Objects) deep learning model, trained and deployed through the Edge Impulse platform for efficient edge inference.

2 Design & Implementation

To develop this project, we followed the steps described inside [1]

2.1 Data collection

To collect the data, two different radars were used: a TrueSense SR250 and an Infineon PWMC radar. To apply beamforming technique and train the object detection network, we decided to focus mainly on the SR250 radar.

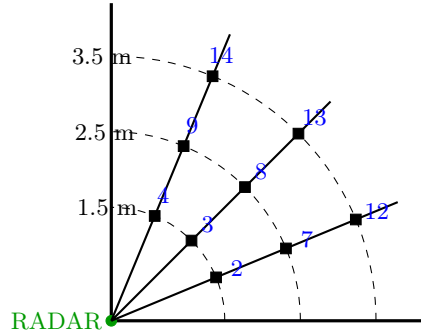


Figure 1: Data collection setup schema

The data collection setup is the one described in Figure 1. Nine positions were considered from the radar location, respectively at a distance of 1.5, 2.5 and 3.5 metres, and considering angles of -22.5° , 0° and 22.5° .

A total of 90 samples were collected during the data collection, with one or more people (up to 4) who were standing in a specific location for 1 minute. Furthermore, in the dataset there are also a couple of samples containing one person moving around different positions every 30 seconds.

For each sample, three NumPy files are saved, each one containing the data for a single receiver, since SR250 has three receiving antennas. Each file stores a NumPy matrix of size (1220, 120), with complex numbers as data type; the first dimension represents the number of time frames, while the second one represents the distance from the radar location as a number of range bins.

2.2 Preprocessing

In radar signal processing, decluttering removes static reflections (clutter) from the environment, thereby enhancing the visibility of moving targets. In this project, we made use of two primary decluttering techniques: Simple Moving Average (SMA) and Exponentially-Weighted Moving Average (EWMA).

The **Simple Moving Average (SMA)** method estimates the background clutter by averaging a fixed number of the most recent radar frames. This technique operates as a **Finite Impulse Response (FIR) filter**, meaning the current background estimate is solely dependent on a finite, predefined window of past inputs. For each radar frame m and range bin k , the estimated background $\tilde{s}_{n,m}[k]$ is calculated by averaging the current frame $s_{n,m}[k]$ and the preceding $W - 1$ frames. The decluttered signal is then obtained by subtracting this average from the current frame:

$$\tilde{s}_{n,m}[k] = s_{n,m}[k] - \frac{1}{W} \sum_{i=m-W+1}^m (s_{n,i}[k])$$

where W is the window size. The output for the first $W - 1$ frames is typically set to zero as a full averaging window is not yet available.

The **Exponentially-Weighted Moving Average (EWMA)** method provides a continuously adapting background estimate by assigning exponentially decreasing weights to older radar frames. This functions as an **Infinite Impulse Response (IIR) filter**, where the current output is influenced by all past inputs, albeit with a diminishing effect over time. The background estimate is initialised with the first radar frame and each subsequent frame i is updated using a “forgetting factor” α :

$$\text{dec_base} = \text{dec_base} \cdot \alpha + \text{cirs}[i, :] \cdot (1 - \alpha)$$

A higher value of α (typically close to 1) results in slower adaptation to changes but offers greater stability for static clutter.

The result is a bidimensional matrix representing the time-distance relation, as depicted in 2. Note that only the distance of the radar is highlighted, indeed, different positions having the same distance from the radar are not distinguished.

2.3 Digital Beamforming

Digital Beamforming (DBF) is a signal processing technique applied to data received from an

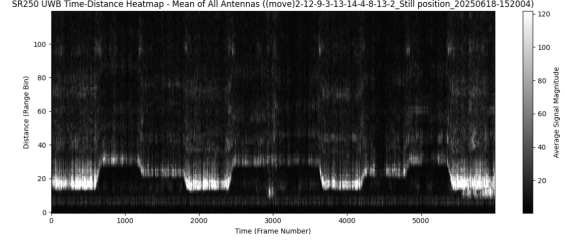


Figure 2: Time-distance plot with a person moving along positions 2-12-9-3-13-14-4-8-13-2

array of antennas to create a spatial filter, effectively steering the array’s sensitivity in a desired direction.

The core principle of DBF involves applying calculated complex weights to the individual signals from multiple receiver antennas before summing them. These weights introduce specific phase shifts and amplitude adjustments. For a linear array of antennas, the phase shift for each element is designed to compensate for the differential propagation delay of a signal arriving from a particular angle. When signals from a target at the desired “beam-steering angle” are coherently summed, their phases align, leading to constructive interference and a strengthened signal. Conversely, signals from other directions will interfere destructively, resulting in their suppression.

In this project we implemented DBF by:

- Utilizing data from two specific receiver antennas (RX0 and RX2) to form a virtual array.
- Calculating beamforming weights for a range of azimuth angles based on the antenna spacing (d), wavelength ($\lambda = c/f$), and wavenumber ($k = 2\pi/\lambda$).
- Applying these weights to the complex time-series data from the selected antennas via a vectorized matrix multiplication to produce a 3D (time, range, angle) beamformed output.

This process transforms the raw time-domain radar measurements into a spatial representation, making it possible to pinpoint the angular location of targets relative to the radar system.

2.4 Object Detection

Following the digital beamforming process, the project aims to implement an object detection system to automatically identify and localize targets within the radar scene. This involves generating a specialised dataset of images from the beamformed radar data and then training a

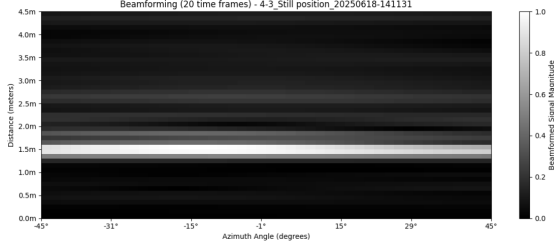


Figure 3: Digital Beamforming on positions 3 and 4

deep learning model for detection.

The foundation for the object detection task lies in the creation of a comprehensive image dataset derived from the 3D beamformed radar data (time, range, angle). The process systematically transforms the continuous radar streams into a sequence of 2D Range-Azimuth images suitable for deep learning.

For the **image generation process**, we followed these steps:

- The full 3D beamformed data, representing radar reflections across time, range bins, and azimuth angles, serves as the input.
- A **sliding time window** approach is employed. For each window (e.g., 40 frames, equivalent to 2 seconds of data, with a stride of 40 frames for non-overlapping windows), the complex data within that window is aggregated.
- **Coherent averaging** (summing complex values before taking the absolute magnitude) is performed across the time frames within each window. This enhances the signal of persistent targets.
- The resulting 2D aggregated data is then **normalized** to a $[0, 1]$ range using min-max scaling, preparing it for visualization and model input.
- These normalized 2D Range-Azimuth maps are saved as grayscale PNG images. Crucially, these images are rendered to a fixed resolution with axes turned off, ensuring a consistent input format for the object detection model.

Bounding Box Labeling: concurrently with image generation, a corresponding ‘.labels’ JSON file is created. For each generated image, bounding box annotations are included, mapping to the known ground truth positions. These ground truth positions, extracted from the original ‘.npz’ filenames (e.g., "12-14_Still position"), are converted into pixel coordinates

(`x`, `y`, `width`, `height`) relative to the fixed image dimensions. A standard bounding box size is used around the center of the projected ground truth location, providing the necessary labels for supervised learning.

2.4.1 Object Detection Model: FOMO Network

For the task of detecting objects within these generated radar images, a specialised deep learning model known as **FOMO (Fast Objects, Mobile Objects)** is employed. FOMO is particularly well-suited for embedded and real-time object detection applications, especially when dealing with smaller objects and lower-resolution imagery, which aligns with the characteristics of the generated radar heatmaps.

FOMO operates by dividing the input image into a grid and predicting whether an object’s centroid falls into each grid cell. This approach is highly efficient for detecting multiple small objects without the overhead of traditional anchor box-based detectors. Its lightweight architecture makes it ideal for deployment on resource-constrained platforms, aligning with potential future applications of UWB radar for embedded sensing. The model is trained on the generated dataset of 2D Range-Azimuth images and their corresponding bounding box labels to learn the spatial patterns indicative of target presence and location.

In this project, the FOMO model was trained and deployed using **Edge Impulse**, which is a leading development platform for machine learning on edge devices. It provides a comprehensive set of tools that streamline the entire machine learning workflow, from raw data collection (in our specific case, we uploaded the data set and the detection of the bounding boxes was done automatically) and preprocessing to model training, optimization and deployment on diverse hardware.

3 Results - TBW

To train the object detection model, we tried several configurations of the model and a different number of items in the dataset. The results shown here are referred to a version in which, from each sampling of 1200 time frames (after the removal of the first, "noisy" samples), windows of size 40 time frames and a step size of 20 are extracted, i.e. the first window extracted for beamforming is in the range $[0, 40]$ time frames, the second windows is in the range $[20, 60]$ and so on. The dataset consists of 5790 images of

size 310x308, splitted between training set and test set in a 80%/20% proportion.

	BACKG	12	13	14	2	3	4	7	8	9
BACKG	100.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
12	6.4%	93.6%	0%	0%	0%	0%	0%	0%	0%	0%
13	30.5%	0%	69.5%	0%	0%	0%	0%	0%	0%	0%
14	1.6%	0%	0%	98.4%	0%	0%	0%	0%	0%	0%
2	34.7%	0%	0%	0%	65.3%	0%	0%	0%	0%	0%
3	11.6%	0%	0%	0%	0%	88.4%	0%	0%	0%	0%
4	8.8%	0%	0%	0%	0%	0%	91.2%	0%	0%	0%
7	16.5%	0%	0%	0%	0%	0%	0%	83.5%	0%	0%
8	14.0%	0%	0%	0%	0%	0%	0%	0%	86.0%	0%
9	6.4%	0%	0%	0%	0%	0%	0%	0%	0%	93.6%
F1 SCO	1.00	0.56	0.42	0.56	0.66	0.60	0.65	0.60	0.58	0.51

Figure 4: Confusion Matrix on the Validation Set

The confusion matrix on the validation set, reported in Figure 4, highlights the predicted labels of the images in the validation set against the actual labels, for the different classes considered in this problem, that are the positions in the data collection.

In the table 1, the values of the metrics (excluding the background label) are shown for FOMO model using **MobileNetV2 0.35**. The quantized version of the model can be uploaded on a microcontroller, considering that:

- If the EON Compiler (RAM-optimized) engine is used, the inferencing time is around 666 ms, with an estimated peak RAM usage of 137.8 KB and a Flash usage of 81.7 KB.
- If the Tensorflow Lite engine is used, the inferencing time is still around 666 ms, with an estimated peak RAM usage of 283.2 KB and a Flash usage of 111 KB.

We compared also with another FOMO model using **MobileNetV2 0.1** as underlying convolutional model. In the table 2, the values of the metrics (excluding the background label) are shown for this FOMO model. The quantized version of the model can be uploaded on a microcontroller, considering that:

- If the EON Compiler (RAM-optimized) engine is used, the inferencing time is around 626 ms, with an estimated peak RAM usage of 124 KB and a Flash usage of 69.3 KB.
- If the Tensorflow Lite engine is used, the inferencing time is still around 626 ms, with an estimated peak RAM usage of 280.7 KB and a Flash usage of 94.9 KB.

4 Discussion

As mentioned before, during the project, we performed several trials both in the beamforming part and in the object detection part.

The first consideration we reached is that the difference between the Simple Moving Average and the Exponentially-Weighted Moving Average is not really appreciable, since the results of the beamforming are similar. For this reason, we mainly followed the Simple Moving Average procedure, which is also the one described in [1].

Another consideration regards the beamforming. For all of us, it was the first time we dealt with this processing technique, and understanding how it works has been a crucial part of the project. Thus, we applied the beamforming for obtaining the range-angle matrix; however, the results we got are not enough, in some cases, to clearly distinguish the "blobs" around the correct position of the target. This could be caused by physical limitations of the specific radar we focused on, issues in the data collection procedure or misunderstanding in applying the beamforming procedure: exploring the limitations and the potential issues of this problem could be the major key to improve the object detection step.

Finally, the object detection part shows that even a model such as FOMO is able to solve the task of object detection using a very constrained hardware, like a microcontroller unit. It may be worthy to notice that no mismatches between the classes are found in the confidence matrix, instead the mismatches always involve the background class and a specific class which represents the position. This happens because the positions inside the images, and so the bounding boxes, are always in the same place, since they refer to the same data collection scenario.

It is worthy noting that different object detection models come with different tradeoffs. In our project, we experimented with MobileNetV2 0.1 and MobileNetV2 0.35: the first model has a lower RAM and Flash usage and a bit smaller inferencing time, however the F1-score, as well as the precision and the recall, are a bit smaller than the FOMO model with MobileNetV2 0.35, which comes at a slightly greater memory consumption and inferencing time.

Conclusions

This project proved to be very interesting for us, as it allowed us to explore the challenges of digital beamforming and object detection in depth. Working on these aspects not only enhanced our technical understanding but also gave us a

FOMO MobileNetV2 0.35	Precision	Recall	F1-Score
Unoptimized (float32)	42.4%	84.4%	56.4%
Optimized (int8)	47.5%	66.1%	48.7%

Table 1: Non-background metrics values for FOMO MobileNetV2 0.35

FOMO MobileNetV2 0.1	Precision	Recall	F1-Score
Unoptimized (float32)	39.8%	86.1%	54.5%
Optimized (int8)	42.9%	56.3%	48.7%

Table 2: Non-background metrics values for FOMO MobileNetV2 0.1

clearer view of the potential of UWB radar technology and the application of object detection on resource-constrained devices such as micro-controllers.

The ethical perspective

Unlike cameras, UWB radar does not capture visual identity (faces, clothing, etc.), which makes it less intrusive and more privacy-preserving. However, it detects presence, movement, and potentially gestures or breathing, so people inside the radar area must be aware and consent to being tracked. Moreover, transparency about the data collection, processing and storage of the data is required; indeed, a potential issue in our project is given by the usage of an external platform (Edge Impulse) to store the acquired data and train the model.

If the object detection algorithm has not been trained on diverse body types (e.g. children, tall people, wheelchair users...), postures, or mobility aids, it might systematically miss or misclassify certain targets, thus fairness in detection must be ensured to avoid exclusion or unintended discrimination.

The market perspective

UWB radar offers privacy-friendlier sensing, works in darkness or through obstacles, and can detect subtle movements (breathing, falls); however they come with a lower resolution than vision systems, may face adoption resistance due to novelty.

Possible applications are:

- **Healthcare and Elderly Care** for non-intrusive fall detection or vital signs monitoring.
- **Smart Homes and Buildings:** e.g. we can adapt the level of lights or air conditioning inside a room depending on the presence of one or more people.
- **Intrusion Detection** without cameras.

- **Child presence detection** inside vehicles based on the presence or the breathing acts, as done in [2] and [3].

Acknowledgements

We thank Prof. Pavan for the patience and the support he gave us during the execution of the project.

References

- [1] G. Repole, “Tinyml-enabled human indoor localization with uwb-radar imaging”, 2023.
- [2] M. Pavan, A. Caltabiano, M. Roveri, *et al.*, “On-device subject recognition in uwb-radar data with tiny machine learning”, in *CEUR Workshop Proceedings*, vol. 3252, 2022, pp. 1–8.
- [3] M. Pavan, A. Caltabiano, and M. Roveri, “Tinyml for uwb-radar based presence detection”, in *2022 International Joint Conference on Neural Networks (IJCNN)*, IEEE, 2022, pp. 1–8.